

Pandas

Pandas

Introduzione alla libreria Pandas per l'analisi dei dati in Python.

Perché è utile usare Pandas

Immagina di avere un foglio Excel con migliaia di righe e di dover trovare tutti gli studenti con media superiore all'8, ordinarli per cognome, e salvare il risultato in un nuovo file. A mano richiederebbe ore.

Con **Pandas** lo fai in poche righe di codice Python — e puoi ripeterlo automaticamente ogni volta che i dati cambiano.

Cos'è Pandas?

Pandas è la libreria Python più usata per lavorare con **dati tabellari** — cioè dati organizzati in righe e colonne, come un foglio Excel o una tabella di database.

Le due strutture principali sono:

- **Series**: una singola colonna di dati (come una lista con etichette)
 - **DataFrame**: una tabella completa con righe e colonne
-

Installazione e importazione

```
pip install pandas
```

Per convenzione, Pandas si importa con il soprannome `pd` :

Series: una colonna di dati

```
# Crea una Series da una lista
voti = pd.Series([8, 7, 9, 6, 10])
print(voti)
# 0      8
# 1      7
# 2      9
# 3      6
# 4     10
# dtype: int64

# Crea una Series con etichette personalizzate (invece dei numeri 0, 1, 2...)
voti_classe = pd.Series(
    [8, 7, 9, 6],
    index=["Alice", "Bob", "Carlo", "Diana"]
)
print(voti_classe["Alice"])    # 8
print(voti_classe.mean())     # 7.5 - media
```

DataFrame: la tabella di dati

Un **DataFrame** è come un foglio Excel: ha righe e colonne con nomi.

```
# Crea un DataFrame da un dizionario (ogni chiave è una colonna)
studenti = pd.DataFrame({
    "nome": ["Alice", "Bob", "Carlo", "Diana"],
    "eta": [16, 15, 17, 16],
    "voto": [8, 7, 9, 6],
    "citta": ["Roma", "Milano", "Roma", "Napoli"]
})

print(studenti)
```

Output:

	nome	eta	voto	citta
0	Alice	16	8	Roma
1	Bob	15	7	Milano
2	Carlo	17	9	Roma
3	Diana	16	6	Napoli

Accedere ai dati

```
# Selezionare una colonna (come accedere a un dizionario)
print(studenti["nome"])

# Selezionare più colonne
print(studenti[["nome", "voto"]])

# Selezionare righe per numero (iloc = integer location)
print(studenti.iloc[0])      # Prima riga (indice 0)
print(studenti.iloc[0:2])    # Righe 0 e 1

# Selezionare un singolo valore (riga, colonna)
print(studenti.iloc[0, 2])   # Riga 0, colonna 2 → 8 (il voto di Alice)
```

Filtrare i dati

Qui Pandas diventa potente: puoi selezionare solo le righe che soddisfano una condizione.

```
# Tutti gli studenti con voto maggiore o uguale a 8
promossi = studenti[studenti["voto"] >= 8]
print(promossi)

# Tutti gli studenti di Roma
romani = studenti[studenti["citta"] == "Roma"]
print(romani)

# Filtri combinati: voto >= 7 E età >= 16
risultato = studenti[(studenti["voto"] >= 7) & (studenti["eta"] >= 16)]
print(risultato)
```

Statistiche in un colpo solo

```
print(studenti["voto"].mean())    # 7.5  - media dei voti
print(studenti["voto"].max())     # 9    - voto più alto
print(studenti["voto"].min())     # 6    - voto più basso
print(studenti["voto"].std())     # deviazione standard

# Riepilogo statistico di tutte le colonne numeriche
print(studenti.describe())
```

Leggere e scrivere file

Una delle funzionalità più utili: Pandas legge e scrive file CSV ed Excel direttamente.

```
# Leggere un file CSV (valori separati da virgole)
df = pd.read_csv("dati.csv")

# Scrivere su CSV (index=False evita di scrivere i numeri di riga)
studenti.to_csv("studenti_export.csv", index=False)

# Leggere un file Excel
df_excel = pd.read_excel("dati.xlsx")

# Scrivere su Excel
studenti.to_excel("studenti_export.xlsx", index=False)
```

Operazioni comuni

```
# Aggiungere una nuova colonna calcolata
studenti["promosso"] = studenti["voto"] >= 6 # True o False per ogni riga
print(studenti)

# Ordinare per voto (dal più alto al più basso)
ordinati = studenti.sort_values("voto", ascending=False)
print(ordinati)

# Raggruppare e calcolare statistiche per gruppo
# (media dei voti per ogni città)
per_citta = studenti.groupby("citta")["voto"].mean()
print(per_citta)

# Informazioni generali sul DataFrame
print(studenti.shape) # (4, 4) - 4 righe, 4 colonne
print(studenti.dtypes) # tipo di dati in ogni colonna
```